Taller — Pilas, colas y listas enlazadas.

Asignatura: Estructura de Datos
Estudiante: LINDA KATRIM CORDOBA HERNANDEZ
Institución Universitaria Pascual Bravo

Parte 1 – Reconocer la estructura adecuada

1. Un editor de texto guarda las acciones recientes para permitir deshacer primero la última acción realizada:

Situación	**Estructura adecuada**	**Justificación concreta**
Un editor guarda acciones recientes para deshacer primero la última acción	**Pila**	La última acción realizada debe deshacerse primero. Eso es exactamente LIFO: *último en entrar, primero en salir*.

Personas esperan su turno y se atiende primero a quien llegó primero	**Cola**	La primera persona que entra a la fila es la primera en ser atendida. Eso es FIFO: *primero en entrar, primero en salir*.
Una aplicación recorre canciones y, al terminar la última, vuelve a la primera	**Lista circular**	El último nodo apunta al primero, por lo que el recorrido no termina en nulo; puede continuar indefinidamente.
Sistema que recorre registros en un solo sentido y agrega con frecuencia al comienzo	**Lista simple**	Permite recorrido en una sola dirección y agregar al inicio en tiempo constante O(1)O(1)O(1), actualizando solo la referencia cabeza.
Reproductor que permite ir a la canción siguiente y a la anterior	**Lista doble**	Cada nodo guarda siguiente y anterior, por lo que se puede avanzar y retroceder desde cualquier canción.

Parte 2 — Pilas (LIFO)

Operación	Resultado	Pila después de la operación
Inicio	Pila vacía	`[]`
`push(4)`	Se agrega 4	Tope → `4`
`push(9)`	Se agrega 9	Tope → `9`, luego `4`
`push(2)`	Se agrega 2	Tope → `2`, luego `9`, luego `4`
`pop()`	Devuelve **2**	Tope → `9`, luego `4`
`push(7)`	Se agrega 7	Tope → `7`, luego `9`, luego `4`
`peek()`	Devuelve **7**, sin eliminarlo	Tope → `7`, luego `9`, luego `4`
`pop()`	Devuelve **7**	Tope → `9`, luego `4`
`pop()`	Devuelve **9**	Tope → `4`

Diagrama final:

TOPE
↓
[4]

Resultados:

- Primer pop() devuelve 2.
- peek() devuelve 7.
- Segundo pop() devuelve 7.
- Tercer pop() devuelve 9.

2.2 Casos límite

Si se hace pop() en una pila vacía

No hay elemento en el tope, por lo que no se puede retirar nada. Esto se llama desbordamiento inferior o *underflow*.

Forma correcta de manejarlo: antes de retirar, verificar si la pila está vacía. Si lo está, se puede devolver un mensaje de error, devolver nulo, o lanzar una excepción controlada.

Si se hace peek() en una pila vacía

peek() no debe modificar la pila, pero tampoco hay un tope que consultar. Lo más seguro es devolver nulo o reportar "pila vacía".

Pseudocódigo de deshacer(pila)

```
FUNCIÓN deshacer(pila)
   SI pila.estaVacia() ENTONCES
      ESCRIBIR "No hay acciones para deshacer"
      DEVOLVER nulo
   FIN SI

   accion ← pila.pop()
   ESCRIBIR "Se deshizo la acción:", accion
   DEVOLVER accion
FIN FUNCIÓN
```

primero se valida el caso vacío. Si hay al menos una acción, pop() retira el elemento del tope, que siempre será la acción más reciente.

2.3 Aplicación: deshacer acciones

El usuario realiza:

1. escribir

2. borrar

3. pegar

Cada acción se apila en el orden en que ocurre.

Pila antes de presionar "deshacer"

TOPE
↓
[pegar] ← última acción realizada
[borrar]
[escribir] ← primera acción realizada

Después del primer “deshacer”

Se deshace la última acción: **pegar**.

TOPE

↓

[borrar]

[escribir]

pop() devuelve: **pegar**.

Después del segundo “deshacer”

Se deshace la siguiente acción más reciente: **borrar**.

TOPE

↓

[escribir]

pop() devuelve: **borrar**.

¿Qué acción se deshace primero?
Se deshace primero **pegar**, porque fue la última acción realizada.

¿Por qué una pila representa este comportamiento?
Porque una pila guarda los elementos en orden LIFO. La última acción entra al tope y, al presionar “deshacer”, sale primero del tope. Así, el sistema deshace las acciones en orden inverso a como fueron realizadas.

Parte 3 — Colas (FIFO)

3.1 Simula la fila de atención

Operación	Resultado	Cola después de la operación
Inicio	Cola vacía	`[]`
`encolar("Ana")`	Ana entra al final	Frente → `Ana` ← Final
`encolar("Luis")`	Luis entra detrás de Ana	Frente → `Ana`, `Luis` ← Final
`encolar("Sol")`	Sol entra al final	Frente → `Ana`, `Luis`, `Sol` ← Final
`desencolar()`	Devuelve **Ana**	Frente → `Luis`, `Sol` ← Final
`encolar("Mia")`	Mia entra al final	Frente → `Luis`, `Sol`, `Mia` ← Final
`desencolar()`	Devuelve **Luis**	Frente → `Sol`, `Mia` ← Final
`encolar("Teo")`	Teo entra al final	Frente → `Sol`, `Mia`, `Teo` ← Final

Diagrama final:

FRENTE FINAL

↓ ↓

[Sol] → [Mia] → [Teo] → nulo

Personas que quedan esperando, en orden de atención:

Sol → Mia → Teo

Personas atendidas:

- Ana fue atendida primero.
- Luis fue atendida segunda.

3.2 Operaciones y casos límite

Pseudocódigo de atender(cola)

```
FUNCIÓN atender(cola)
    SI cola.estaVacia() ENTONCES
        ESCRIBIR "No hay personas en la fila"
```

```
        DEVOLVER nulo
    FIN SI

    persona ← cola.desencolar()
    ESCRIBIR "Atendiendo a:", persona
    DEVOLVER persona
FIN FUNCIÓN
```

Si se intenta desencolar una cola vacía

No hay nadie en el frente, por lo que no se puede atender a nadie. Debe validarse antes:

```
SI cola.estaVacia() ENTONCES
    DEVOLVER nulo
FIN SI
```

¿Por qué agregar al frente y retirar desde el final no representa correctamente una fila FIFO?

Porque en una fila real, quien llega primero debe salir primero. Si se agrega por el frente, el nuevo elemento queda delante de todos los que ya estaban esperando. Si además se retira desde el final, se atendería primero a la persona que llegó de último.

Eso invierte el orden FIFO y se comporta más como una pila LIFO:

FIFO correcto:

Entrada → [Ana, Luis, Sol] → Salida

Comportamiento incorrecto propuesto:

Entrada por el frente → [Sol, Luis, Ana]

Salida desde el final → primero sale Ana

Parte 4 — Listas enlazadas simples

4.1 Construye y modifica una lista

Operación 1: insertar 18 al final

cabeza → [18 | nulo]

Operación 2: insertar 7 al comienzo

El nuevo nodo 7 apunta al antiguo primer nodo, que era 18. Luego, cabeza apunta a 7.

cabeza → [7 | •] → [18 | nulo]

Operación 3: insertar 24 al final

cabeza → [7 | •] → [18 | •] → [24 | nulo]

Operación 4: insertar 13 después del nodo que contiene 7

El nodo 13 debe apuntar al nodo 18. El nodo 7 debe apuntar al nodo 13.

cabeza → [7 | •] → [13 | •] → [18 | •] → [24 | nulo]

Operación 5: eliminar el nodo que contiene 18

El nodo 13 debe dejar de apuntar a 18 y comenzar a apuntar a 24.

cabeza → [7 | •] → [13 | •] → [24 | nulo]

Lista final: 7 → 13 → 24 → nulo

Referencia actualizada al eliminar 18:
nodo13.siguiente = nodo24

4.2 Diseña operaciones

Recorrer la lista e imprimir todos sus datos

FUNCIÓN recorrerLista(cabeza)

```
  actual ← cabeza

  MIENTRAS actual ≠ nulo HACER
    ESCRIBIR actual.dato
    actual ← actual.siguiente
  FIN MIENTRAS
FIN FUNCIÓN
```

se comienza en la cabeza. En cada repetición se imprime el dato y se avanza al nodo siguiente. El ciclo termina cuando actual llega a nulo.

Buscar un valor y devolver si está presente

```
FUNCIÓN buscar(cabeza, valor)
  actual ← cabeza

  MIENTRAS actual ≠ nulo HACER
    SI actual.dato = valor ENTONCES
      DEVOLVER verdadero
    FIN SI

    actual ← actual.siguiente
  FIN MIENTRAS

  DEVOLVER falso
FIN FUNCIÓN
```

Insertar un nodo al comienzo

```
FUNCIÓN insertarAlComienzo(cabeza, valor)
  nuevoNodo ← crear Nodo(valor)
  nuevoNodo.siguiente ← cabeza
  cabeza ← nuevoNodo
  DEVOLVER cabeza
FIN FUNCIÓN
```

4.3 Comprueba los bordes

Cuando la lista está vacía

Antes:
cabeza → nulo

Después de insertar 5:

cabeza → [5 | nulo]

Cuando la lista ya tiene uno o más nodos
Antes:
cabeza → [7 | •] → [13 | nulo]

Después de insertar 5 al comienzo:
cabeza → [5 | •] → [7 | •] → [13 | nulo]

Eliminar el primer nodo

Supongamos esta lista:

Antes:

cabeza → [7 | •] → [13 | •] → [24 | nulo]

Para eliminar 7, la cabeza debe apuntar al segundo nodo, es decir, al nodo 13.

Después:

cabeza → [13 | •] → [24 | nulo]

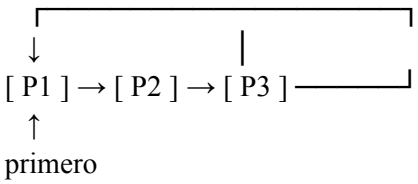
Referencias que deben actualizarse:

cabeza ← cabeza.siguiente

Parte 5 — Listas circulares

5.1 Reproduce un recorrido circular

Lista circular inicial



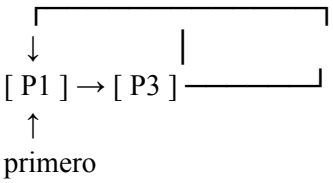
El nodo P3 apunta de nuevo a P1.

Registro de los primeros siete turnos

P2 termina en el segundo turno que recibe.

Turno	Proceso atendido	Situación después del turno	Siguiente proceso
1	P1	P1 sigue activo	P2
2	P2	P2 termina	P3
3	P3	P3 sigue activo	P1
4	P1	P1 sigue activo	P3
5	P3	P3 sigue activo	P1
6	P1	P1 sigue activo	P3
7	P3	P3 sigue activo	P1

Lista después de retirar P2



Verificación: P3, que ahora es el último nodo, apunta nuevamente a P1, el primer nodo. La circularidad se conserva.

Condición para detener un recorrido de todos los nodos

Como nunca se llega a nulo, no se puede usar la condición actual ≠ nulo. Se debe guardar el nodo inicial y detenerse cuando el recorrido vuelva a él:

actual ← primero

repetir

    procesar actual

    actual ← actual.siguiente

hasta que actual = primero

Condición de parada: actual = primero, después de haber avanzado al menos una vez.

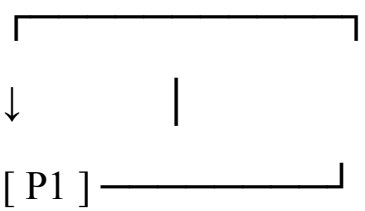
5.2 Casos especiales

Caso 1: la lista está vacía

primero → nulo

Caso 2: la lista contiene un solo nodo

Antes:



primero

Al eliminar P1, la lista queda vacía:

Después:

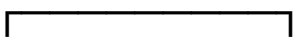
primero → nulo

Enlaces a actualizar: primero ← nulo.

Caso 3: se elimina el nodo considerado primero, con dos o más nodos

Supongamos:

Antes:





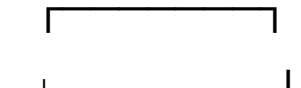
[P1] → [P2] → [P3]



primero

Se elimina P1. El nuevo primero es P2 y P3 debe apuntar a P2.

Después:



[P2] → [P3]



primero

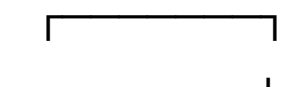
Enlaces a actualizar:

primero ← primero.siguiente

ultimo.siguiente ← primero

Caso 4: se elimina el último nodo

Antes:



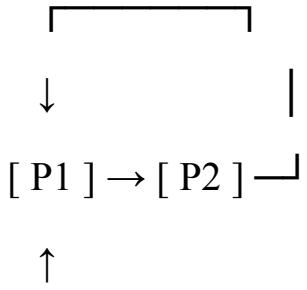
[P1] → [P2] → [P3]



primero

Se elimina P3. Ahora P2 es el último nodo y debe apuntar a P1.

Después:



Enlaces a actualizar:

nodoAnteriorAlUltimo.siguiente ← primero

Si se guarda una referencia al último nodo, también debe actualizarse:

ultimo ← nodoAnteriorAlUltimo

Parte 6 — Listas doblemente enlazadas

6.1 Recorrido en ambos sentidos

Lista construida

nulo ← [Lunes] ⇄ [Martes] ⇄ [Miércoles] ⇄ [Jueves] → nulo

Representación detallada de cada nodo

Nodo 1:

anterior = nulo

dato = Lunes

siguiente = Martes

Nodo 2:

anterior = Lunes

dato = Martes

siguiente = Miércoles

Nodo 3:

anterior = Martes

dato = Miércoles

siguiente = Jueves

Nodo 4:

anterior = Miércoles

dato = Jueves

siguiente = nulo

Recorrido desde el comienzo hasta el final

Lunes → Martes → Miércoles → Jueves

Recorrido desde el final hasta el comienzo

Jueves → Miércoles → Martes → Lunes

6.2 Inserta y elimina

Lista inicial

nulo ← [Lunes] ⇄ [Martes] ⇄ [Miércoles] ⇄ [Jueves] → nulo

Insertar Domingo antes de Lunes

Domingo se convierte en el nuevo primer nodo.

nulo ← [Domingo] ⇄ [Lunes] ⇄ [Martes] ⇄ [Miércoles] ⇄ [Jueves] → nulo

Referencias actualizadas:

Domingo.anterior = nulo

Domingo.siguiente = Lunes

Lunes.anterior = Domingo

Insertar Viernes después de Jueves

Viernes se convierte en el nuevo último nodo.

nulo ← [Domingo] ⇄ [Lunes] ⇄ [Martes] ⇄ [Miércoles] ⇄ [Jueves] ⇄ [Viernes] → nulo

Referencias actualizadas:

Jueves.siguiente = Viernes

Viernes.anterior = Jueves

Viernes.siguiente = nulo

Eliminar Miércoles

Miércoles está entre Martes y Jueves. Para eliminarlo:

- Martes debe apuntar hacia Jueves.
- Jueves debe apuntar hacia Martes.

Antes:

[Martes] ⇄ [Miércoles] ⇄ [Jueves]

Después:

[Martes] ⇄ [Jueves]

Lista final:

nulo ← [Domingo] ⇄ [Lunes] ⇄ [Martes] ⇄ [Jueves] ⇄ [Viernes] → nulo

Referencias que cambian:

Martes.siguiente = Jueves

Jueves.anterior = Martes

El nodo Miércoles queda desconectado de la lista.

6.3 Compara los enlaces

Una ventaja de la lista doble es que permite recorrer hacia atrás sin necesidad de buscar el nodo anterior desde el inicio. Por ejemplo, desde Jueves se puede ir directamente a Miércoles, luego a Martes y finalmente a Lunes. En una lista simple esto no sería posible directamente, porque cada nodo solo conoce a su siguiente.

El costo es que cada nodo necesita una referencia adicional, anterior. Esto consume más memoria y hace que las inserciones y eliminaciones requieran actualizar más enlaces.

Parte 7 — Análisis y elección

Estructura y representación	Operación	Complejidad	Justificación breve
Pila enlazada con referencia al tope	Insertar en el tope	**O(1)**	Solo se crea el nodo, se hace apuntar al antiguo tope y se actualiza la referencia `tope`.
Pila enlazada con referencia al tope	Retirar del tope	**O(1)**	Solo se guarda el dato del tope, se mueve `tope` al nodo siguiente y se desconecta el nodo anterior.
Cola enlazada con referencias al frente y al final	Encolar al final	**O(1)**	Se tiene acceso directo al último nodo, por lo que no hay que recorrer toda la cola.
Cola enlazada con referencias al frente y al final	Desencolar del frente	**O(1)**	Se retira el nodo apuntado por `frente` y se actualiza `frente` a `frente.siguiente`.
Lista simple sin referencia al último nodo	Insertar al comienzo	**O(1)**	El nuevo nodo apunta a la antigua cabeza y `cabeza` apunta al nuevo nodo.
Lista simple sin referencia al último nodo	Insertar al final	**O(n)**	Hay que recorrer toda la lista desde la cabeza hasta encontrar el último nodo.
Lista doble con referencia a un nodo conocido	Desconectar ese nodo	**O(1)**	Como se conoce el nodo, se actualizan directamente los enlaces de su nodo anterior y su nodo siguiente.

¿Por qué una cola con solo frente tardaría más en encolar al final?

Porque, para agregar al final, el algoritmo debería comenzar en frente y avanzar nodo por nodo hasta encontrar el último. Si la cola tiene nnn elementos, ese recorrido cuesta O(n)O(n)O(n).

En cambio, si la cola también guarda final, puede conectar el nuevo nodo directamente después del último:

final.siguiente = nuevoNodo

final = nuevoNodo

Eso requiere una cantidad fija de pasos, por lo que es O(1)O(1)O(1).

Ventaja de la lista circular para volver repetidamente al primer elemento

La lista circular permite que el último elemento apunte al primero. Así, al llegar al final, el recorrido puede continuar automáticamente desde el inicio sin buscar nuevamente la cabeza ni reiniciar manualmente el recorrido.

Por ejemplo, en una lista de canciones:

Canción 1 → Canción 2 → Canción 3 → Canción 1

Parte 8 — Reflexión

La forma en que se conectan los nodos determina el orden en que los datos pueden insertarse, retirarse y recorrerse. En una **pila**, todos los movimientos ocurren por el tope, por lo que el último dato agregado es el primero en salir; esto es útil para deshacer acciones. En una **cola**, los datos entran por el final y salen por el frente, respetando el orden de llegada, como en una fila de atención. En una **lista simple**, los nodos avanzan en una sola dirección mediante siguiente; en una **lista circular**, el último nodo apunta al primero, permitiendo recorridos continuos; y en una **lista doble**, cada nodo conoce tanto a su anterior como a su siguiente, lo que permite avanzar y retroceder. Por eso, elegir la estructura correcta depende del comportamiento que necesita la aplicación, no solamente de la cantidad de datos que deba guardar.